

Entwicklung eines didaktischen Konzepts zur Einführung in die Mikrocontroller-Programmierung unter Verwendung des Pololu 3pi+ 2040

Santiago Ardiel Cabrera Schmitt
Matrikelnummer: 2218003

Betreuer: Prof. Dr.-Ing. Alexander Biedermann

23. November 2025

Inhaltsverzeichnis

1	Kontextualisierung der Lehrinhalte: Mikrocontroller-Programmierung im hochschulischen Unterricht	4
1.1	Hintergrund: Micromouse-Wettbewerb	4
1.2	Ziel der Arbeit	5
1.3	Aufbau der Arbeit	5
2	Konzeption und Aufbau der Versuchsumgebung	5
2.1	Funktionale Anforderungen und Auswahl des Pololu 3pi+	5
2.2	Layout und Fertigung des Labyrinths	6
2.3	Peripherie und sonstige Hardware	7
3	Erstellung der Übungsblätter	8
3.1	Übungsblatt 1: Einführung in die Programmierung des Roboters	8
3.2	Übungsblatt 2: Arbeiten mit Sensoren und Motorsteuerung	10
3.3	Übungsblatt 3: Implementierung eines Algorithmus zur Labyrinthlösung	16
4	Zukünftiges Konzept für den Aufbau einer Micromouse-Infrastruktur	26

Glossar und Abkürzungsverzeichnis

DFS Depth-First Search. 17

IDE Integrated Development Environment. 6, 8

IMU inertial measurement unit. 20

IR Infrarot. 13

MEMS Mikro-Elektromechanisches System. 18

ToF Time-of-Flight. 13

1 Kontextualisierung der Lehrinhalte: Mikrocontroller-Programmierung im hochschulischen Unterricht

1.1 Hintergrund: Micromouse-Wettbewerb

Der Micromouse-Wettbewerb wurde etwa in den 1970er-Jahren ins Leben gerufen und hat sich mittlerweile weltweit etabliert, wobei er insbesondere in den USA, England und Japan große Beliebtheit genießt [1]. In diesem Wettbewerb entwickeln die Teilnehmenden, meist Studierende technischer Hochschulen, einen autonomen Roboter, der ein Labyrinth in möglichst kurzer Zeit durchqueren soll [2]. Zu den häufig verwendeten Lösungsansätzen zählen der Flood-Fill-Algorithmus, der Dijkstra-Algorithmus sowie verschiedene Suchalgorithmen, wie sie auch für Graphen und Bäume verwendet werden. In dieser Arbeit wird der Backtracking-Algorithmus eingesetzt. Zwar liefert dieser nicht zwangsläufig den schnellsten Lösungsweg, jedoch lässt er sich im Rahmen der Übungseinheiten aufgrund seiner überschaubaren Komplexität gut umsetzen. Algorithmen, die das gesamte Labyrinth analysieren, sind zwar oft effizienter, jedoch auch deutlich komplexer in der Implementierung. Das Labyrinth des offiziellen Wettbewerbs besteht aus 16×16 Zellen, wobei jede Zelle eine Größe von $18\text{ cm} \times 18\text{ cm}$ aufweist. Die weißen Wände sind 5 cm hoch und $1,2\text{ cm}$ dick. Der Startpunkt befindet sich in einer der vier Ecken, während sich das Ziel in der Mitte des Spielfelds befindet (vgl. Abbildung 1.1).

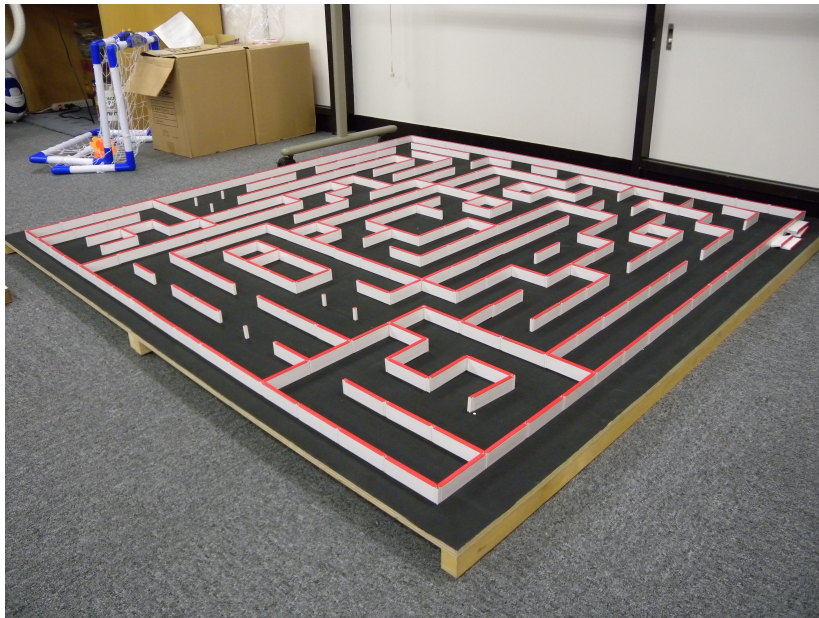


Abbildung 1.1: Micromouse-Labyrinth [3]

1.2 Ziel der Arbeit

Im Rahmen dieser Studienarbeit sollen Studierende in das Konzept der Mikrocontroller-Programmierung eingeführt werden. Dazu implementieren sie einen Backtracking-Algorithmus zur Lösung eines Labyrinths, inspiriert durch die Micromouse-Wettbewerbe. Die Studierenden arbeiten mit einem Raspberry Pi und wenden die grundlegenden Prinzipien der Programmierung anhand eines praktischen Beispiels an.

Darüber hinaus sollen sie den bereitgestellten Code verstehen und kritisch analysieren, um zu beurteilen, inwieweit dieser auf reale Systeme übertragbar ist. Auf dieser Basis entwickeln sie eigene Lösungsansätze, um die Ergebnisse zu validieren und die regelungstechnische Verarbeitung der Daten zu optimieren.

1.3 Aufbau der Arbeit

Zur Vermittlung dieser Konzepte werden die Studierenden in drei aufeinander aufbauenden Übungsblättern zunächst in die Arduino-Programmierungsumgebung eingeführt. In den Übungen werden sie schrittweise an die verschiedenen Sensoren und Aktoren des Pololu 3pi+ [4] herangeführt. Dabei bearbeiten sie konkrete Anwendungsfälle für jedes Steuerungselement und sollen die erzielten Ergebnisse gezielt untersuchen, überprüfen und begründet bewerten.

Im Rahmen des übergeordneten Projekts, ein Labyrinth zu lösen, entwickeln die Studierenden zunehmend ein Verständnis für die Funktionsweise der genutzten Komponenten und setzen ihr Wissen in neuen Kontexten ein. Die Erkenntnisse aus den einzelnen Aufgaben führen sie schrittweise dazu, am Ende eigenständig ein Labyrinth mithilfe eines Backtracking-Algorithmus zu lösen, wobei sie die zuvor gewonnenen Einsichten anwenden, reflektieren und nachweisen können.

2 Konzeption und Aufbau der Versuchsumgebung

Da an der Technischen Hochschule Aschaffenburg bislang keine Vorarbeiten zu einem allgemeinen Konzept für Micromouse-Projekte existierten, war es zunächst notwendig, ein grundlegendes Rahmenkonzept zu entwickeln. Dieses gliederte sich in drei Teilbereiche: die Auswahl eines geeigneten Roboters, das Design des Labyrinths sowie die Zusammenstellung aller benötigten Materialien zur Umsetzung.

2.1 Funktionale Anforderungen und Auswahl des Pololu 3pi+

Da die eigenständige Entwicklung eines Roboters den zeitlichen Rahmen von drei vorgesehenen Übungsblättern deutlich überschreiten würde, wurde ein vorkonfiguriertes System benötigt, das die wesentlichen funktionalen Anforderungen zur Lösung eines Labyrinths erfüllt. Für eine autonome Navigation sind insbesondere Sensoren

wie ein Gyroskop, Infrarot-Abstandssensoren sowie Encoder an den Antriebsmotoren von zentraler Bedeutung. Diese Komponenten ermöglichen es den Studierenden, Bewegungsdaten und Umgebungsinformationen systematisch zu erfassen, einzuordnen und für eine spätere regelungstechnische Auswertung zu nutzen.

Zur Reduktion des Implementierungsaufwands fiel die Wahl auf den Pololu 3pi+ 2040. Dieser Roboter bietet eine Vielzahl integrierter Sensoren und Aktoren, eine gut dokumentierte Programmierschnittstelle sowie eine breite Kompatibilität mit der Arduino Integrated Development Environment (IDE). Die Plattform eignet sich dadurch ideal für eine didaktische Aufbereitung, da sie es den Studierenden erlaubt, technische Zusammenhänge nachvollziehbar zu verstehen, verschiedene Steuerungs- und Regelungsansätze anzuwenden und die Auswirkungen ihrer Entscheidungen reflektiert zu bewerten. Damit bildet der Roboter eine geeignete Grundlage für ein kompaktes, praxisorientiertes Laborkonzept.

2.2 Layout und Fertigung des Labyrinths

Im Rahmen der Etablierung eines Micromouse-Konzepts an der Hochschule müssen neben dem Roboter selbstverständlich auch die Labyrinthentworfen und gefertigt werden. Ein vollständiges Labyrinth bietet jedoch aufgrund seiner Komplexität keine geeignete Grundlage für den Lernprozess, da die Zielsuche wesentlich mehr Zeit in Anspruch nimmt und somit schnelle Tests erschwert. Auch aus Platzgründen ist ein großes Labyrinth für den Einsatz in Vorlesungssälen ungeeignet. Sollte in Zukunft ein größeres Interesse am Wettbewerb entstehen und entsprechend mehr Raum zur Verfügung stehen, kann zu einem späteren Zeitpunkt ein offizielles Labyrinth hergestellt oder angeschafft werden.

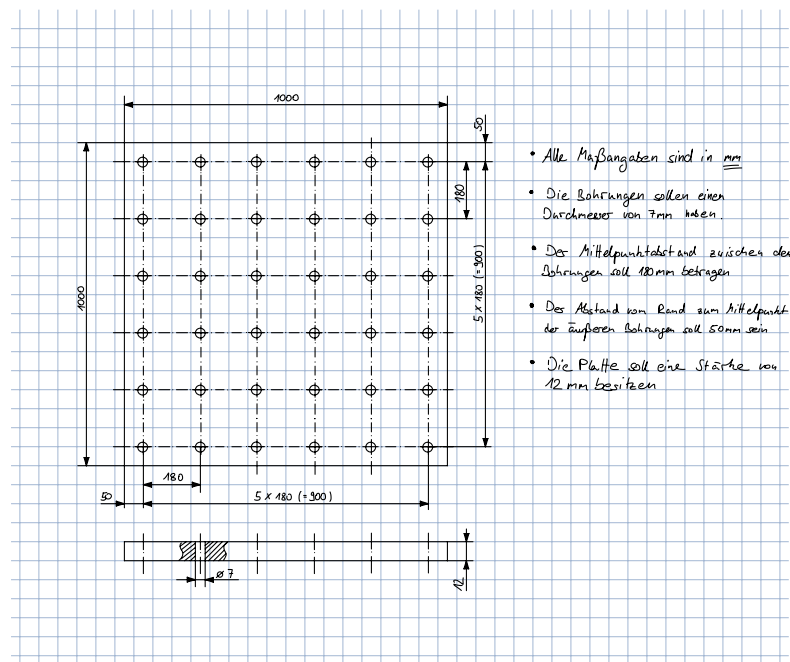
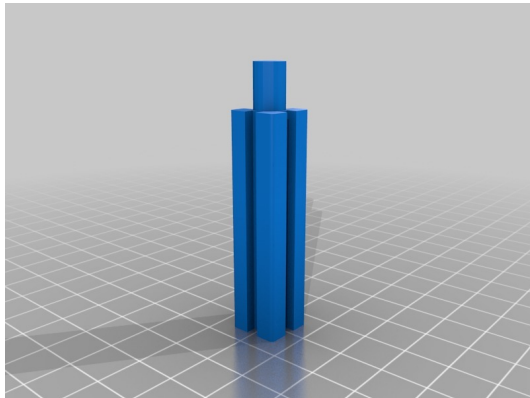
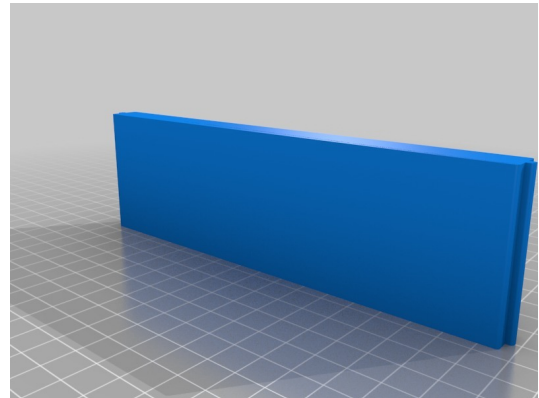


Abbildung 2.1: Konstruktionszeichnung des entworfenen 5×5 -Labyrinth

Unter Berücksichtigung dieser Aspekte wurde ein 5×5 -Labyrinth entworfen, dessen Felder mit den originalgetreuen Abmessungen von $18\text{ cm} \times 18\text{ cm}$ ausgeführt sind. An den notwendigen Stellen wurden Bohrungen vorgesehen, um den modularen Aufbau zu ermöglichen. Das fertige Design ist in Abbildung 2.2 dargestellt. Als Material wurde MDF gewählt wegen seiner glatten homogenen Oberfläche. In diese Bohrungen werden die in Abbildung 2.2a gezeigten Pfosten eingesteckt, zwischen denen anschließend die passenden Wände (vgl. Abbildung 2.2b) positioniert werden. Die STL-Dateien sind unter dieser Quelle [5] zu finden, jedoch müssen die Wände um 2 mm verkürzt werden, um zur Struktur des Labyrinths zu passen. Auf diese Weise kann das Labyrinth beliebig verändert und an unterschiedliche Übungsszenarien angepasst werden.



(a) Steckbarer Pfosten



(b) Steckbare Wände

Abbildung 2.2: Druckbare Wände und Pfosten zum modularen Aufbau des Labyrinths [5]

Das Labyrinth sollte durch das Zusammenstecken mit einem zweiten erweitert werden können. Dies wurde bei der Konstruktion jedoch nicht bedacht, weshalb eine schmale Holzplatte zwischen beide Labyrinth gelegt werden muss.

2.3 Peripherie und sonstige Hardware

Für den Betrieb der Roboter und die Fertigung des Labyrinths war die Beschaffung zusätzlicher Materialien und Hardware notwendig, welche in diesem Abschnitt kurz aufgeführt werden. Für jeden Roboter wurden jeweils zwei Sätze aus vier NiMH-Akkus bestellt, um sicherzustellen, dass die Studierenden die Roboter jederzeit ohne Ausfallzeiten verwenden können. Zum Laden der Akkus wurden zudem ausreichend Ladestationen angeschafft.

Für den Druck der Labyrinthwände stand im Labor bereits ein 3D-Drucker zur Verfügung, für den zusätzlich ausreichend weißes Filament beschafft wurde. Die Wahl der Farbe fiel auf Weiß, da dieses Material die Infrarotlichtimpulse des Infrarotsensors am besten reflektiert. Im Rahmen des Druckvorgangs wurde eine Stromkostenberechnung durchgeführt, deren Ergebnisse zusammen mit einer Gesamtkostenaufstellung an die Labormitarbeitenden übermittelt wurden.

3 Erstellung der Übungsblätter

Die Einführung in die Programmierung des Pololu 3pi+ erfolgt im Rahmen von drei aufeinander aufbauenden Übungsblättern. In jeder Einheit werden die Studierenden schrittweise mit den zentralen Sensoren und Steuerungselementen des Roboters vertraut gemacht. Der Fokus liegt dabei nicht nur auf der Umsetzung der Programmierung, sondern insbesondere darauf, die Wechselwirkungen zwischen Software und Hardware zu verstehen und einzuordnen.

Durch gezielte Aufgabenstellungen sollen die Studierenden erkennen, dass ein theoretisch korrektes Programm in der praktischen Umsetzung nicht zwangsläufig fehlerfrei funktioniert. Typische Herausforderungen wie Messungenauigkeiten, Verzögerungen im System oder mechanische Abweichungen treten bewusst zutage und müssen von den Studierenden analysiert und berücksichtigt werden. Dadurch entsteht ein Verständnis für die Grenzen rein softwarebasierter Überlegungen, und die Notwendigkeit regelungstechnischer Anpassungen wird klarer.

In der letzten Übung übertragen die Studierenden ihr erarbeitetes Wissen auf eine komplexere Aufgabenstellung und lassen den Roboter ein Labyrinth lösen. Dabei wenden sie ihre Erkenntnisse an, reflektieren deren Wirkung im Gesamtsystem und bewerten die Funktionsfähigkeit ihrer Lösung.

3.1 Übungsblatt 1: Einführung in die Programmierung des Roboters

In der ersten Übung liegt der Schwerpunkt auf der Einführung in die Arduino-Entwicklungsumgebung (IDE) sowie dem Verständnis ihrer grundlegenden Funktionalitäten. Als erste praktische Anwendung implementieren die Studierenden ein klassisches Blink-Programm, welches eine LED im Sekundentakt ein- und ausschaltet.

Die Wahl der Arduino IDE erfolgte aufgrund ihrer besonderen Eignung für den didaktischen Einsatz: Sie bietet eine benutzerfreundliche Plattform, die den Einstieg in die Mikrocontroller-Programmierung erleichtert und zugleich über eine Vielzahl nützlicher Werkzeuge verfügt. So gewährleistet der integrierte Boards Manager, dass beim Kompilieren die korrekte Zielarchitektur berücksichtigt wird, während der Library Manager die unkomplizierte Installation zusätzlicher Bibliotheken ermöglicht. Für den Pololu 3pi+ stehen darüber hinaus spezifische Bibliotheken bereit, die einen unmittelbaren Zugriff auf die vorhandenen Sensoren und Aktoren erlauben und somit die Komplexität der Hardwareansteuerung abstrahieren. Darüber hinaus stellt die Arduino IDE mit dem seriellen Monitor und dem seriellen Plotter hilfreiche Werkzeuge zur Verfügung, die es den Studierenden gestatten, Variablenwerte und Systemzustände in Echtzeit auszugeben und zu beobachten.

Für die praktische Umsetzung dieser Einführung wird den Studierenden ein Link zum GitHub-Repository von Pololu bereitgestellt. Dieses enthält eine detaillierte Anleitung zur Installation des erforderlichen Boards sowie eine exemplarische Demonstration des Upload-Prozesses eines Sketches. Zwar wird das Hochladen von Programmen im Übungsblatt nochmals ausführlicher erläutert, jedoch soll an dieser Stelle insbesondere die Bedeutung von GitHub als zentrale Ressource hervorgehoben werden. Viele Hardwarehersteller stellen dort ihre Software-Bibliotheken und Schnittstellen zur Verfügung. Damit wird den Studierenden der Zugang zu praxisrelevanten Entwicklungswerkzeugen eröffnet, die in den folgenden Übungen eine wesentliche Rolle spielen.

Nachdem die Entwicklungsumgebung eingerichtet wurde, erstellen die Studierenden ihren ersten Sketch. Dabei wird der grundlegende Aufbau eines Arduino-Programms vermittelt, das sich aus zwei zentralen Funktionsblöcken zusammensetzt: der `setup`-Funktion, die beim Programmstart einmalig ausgeführt wird und der Initialisierung von Hardwarekomponenten dient, sowie der `loop`-Funktion, welche in einer Endlosschleife abläuft. Im ersten Sketch soll eine LED im Sekundentakt ein- und ausgeschaltet werden. Um den Einstieg zu erleichtern, werden die hierfür notwendigen Standardfunktionen der Arduino IDE zur Verfügung gestellt. Zudem wird die Pin-Nummer der integrierten Benutzer-LED des Pololu 3pi+ 2040 [4] angegeben. Zum Einschalten der LED muss der entsprechende Pin auf `LOW` gesetzt werden. Auf diesen zunächst unintuitiven Umstand wird durch einen Hinweis verwiesen, ohne die Lösung unmittelbar vorwegzunehmen, sodass die Studierenden die logischen Zusammenhänge selbstständig erkennen können.

In der letzten Aufgabe des Übungsblattes sollen die Studierenden mit den Debugging-Tools der Arduino IDE sowie mit dem integrierten OLED-Display vertraut gemacht werden, das sich hervorragend für Diagnose- und Testzwecke eignet. Dabei soll der Zustand des LED-Pins sowohl über den seriellen Monitor als auch über das OLED-Display ausgegeben und kontrolliert werden. Zu beachten ist hierbei, dass in der IDE und im Programmcode mittels der Funktion `Serial.begin(long baud)` dieselbe Baudrate eingestellt ist und im Editor der korrekte serielle Port ausgewählt wurde. Hier wurde bewusst auf eine detaillierte Anleitung für das OLED-Display verzichtet und stattdessen auf die offizielle Dokumentation verwiesen, um das selbstständige Recherchieren in externen Schnittstellenbeschreibungen zu fördern.

Die Musterlösung zu dieser sowie zu den folgenden Aufgaben folgt einem einheitlichen Schema. Zunächst wird in einer kurzen Einleitung auf die Herangehensweise an die jeweilige Problemstellung eingegangen und auf relevante Dokumentationen verwiesen. Während in der ersten Übung diese Verweise bereits in der Aufgabenstellung enthalten sind, ist dies in den nachfolgenden Übungen nicht immer der Fall. Durch die gezielte Verlinkung innerhalb der Musterlösung erhalten die Studierenden die Möglichkeit, nachträglich auf die benötigten Ressourcen zuzugreifen und die Aufgabenstellung eigenständig zu lösen, falls dies zuvor nicht gelungen ist. Im Anschluss folgt der eigentliche Programmcode, in dem neu eingeführte Funktionen sowie komplexere Abschnitte durch Kommentare erläutert werden. Teilweise werden auch alternative Lösungsansätze aufgezeigt, um Studierenden, die eigene Herangehensweisen entwickelt haben, einen Vergleich zu ermöglichen und zu verdeutlichen, dass es in der Softwareentwicklung meist mehrere valide Lösungswege gibt.

3.2 Übungsblatt 2: Arbeiten mit Sensoren und Motorsteuerung

Während das erste Übungsblatt eine einführende Orientierung zum Pololu 3pi+ 2040 Roboter bietet, liegt der Schwerpunkt der zweiten Übung auf der detaillierten Analyse der Sensorik und Aktorik. Die Studierenden untersuchen die Funktionsweise der einzelnen Komponenten und evaluieren relevante Dokumentationen, um ein tiefgehendes Systemverständnis zu entwickeln. In der Einleitung wird der Umfang der Übung skizziert und explizit auf die Dokumentation des Infrarotsensors verwiesen, welche aufgrund der Nutzung einer spezifischen Bibliothek gesondert behandelt wird.

In der ersten Aufgabe wird mit dem Encoder der erste Aktor technisch eingeführt. Auf der Platine des Pololu 3pi+ 2040 sind zwei Hall-Effekt-Sensoren in Quadratur verschaltet. Ein Hall-Effekt-Sensor beruht auf dem physikalischen Hall-Effekt. Dieser tritt auf, wenn ein stromdurchflossener Leiter oder Halbleiter einem senkrecht einwirkenden Magnetfeld ausgesetzt wird. Durch die Lorentzkraft werden die bewegten Ladungsträger seitlich abgelenkt, wodurch sich auf den gegenüberliegenden Seiten des Leiters unterschiedliche Ladungsdichten bilden (schematisch in Abbildung 3.1 dargestellt). Zwischen diesen Seiten entsteht eine messbare Potentialdifferenz, die als Hall-Spannung bezeichnet wird. Im Sensor wird diese Spannung erfasst, um Änderungen der Magnetfeldstärke oder -richtung zu detektieren.

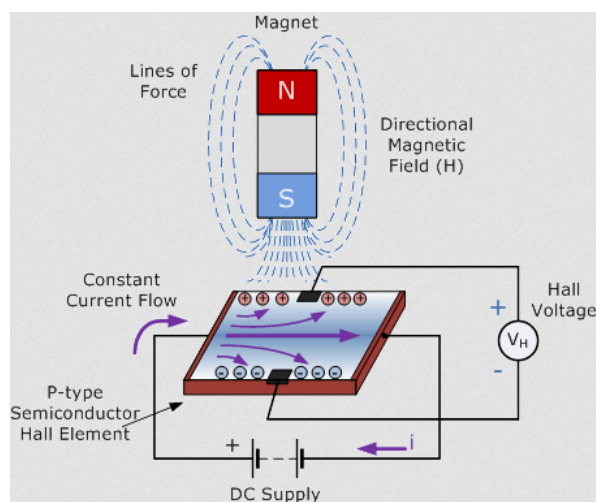


Abbildung 3.1: Hall-Effekt [6]

Im vorliegenden Anwendungsfall ist die Motorwelle verlängert und mit einer Magnetscheibe versehen, die über sechs magnetische Pole verfügt (vgl. Abbildung 3.2a). Gegenüberliegend ist das Getriebe montiert. Die Drehbewegung der Magnetscheibe wird von zwei Hall-Effekt-Sensoren erfasst, die in Quadratur, also um 90° phasenversetzt, angeordnet sind. Durch diese Anordnung kann der Encoder nicht nur die Anzahl der Impulse, sondern auch die Drehrichtung des Motors eindeutig bestimmen.

Bevor die Messwerte der Encoder zur Bestimmung der Radbewegung verwendet werden können, muss die Anzahl der erfassten Impulse mit dem Übersetzungsverhältnis des Getriebes verrechnet werden. Bei einer Magnetscheibe mit sechs Polen ergibt sich eine Auflösung von zwölf Zählimpulsen pro Umdrehung der Motorwelle. Unter Berücksichtigung des Getriebeübersetzungsverhältnisses von 30:1 resultiert daraus eine effektive Auflösung von $30 \times 12 = 360$ Zählimpulsen pro Radumdrehung.

In Abbildung 3.2b ist exemplarisch eine externe Platine mit zwei Hall-Effekt-Sensoren in Quadratur-Schaltung dargestellt.

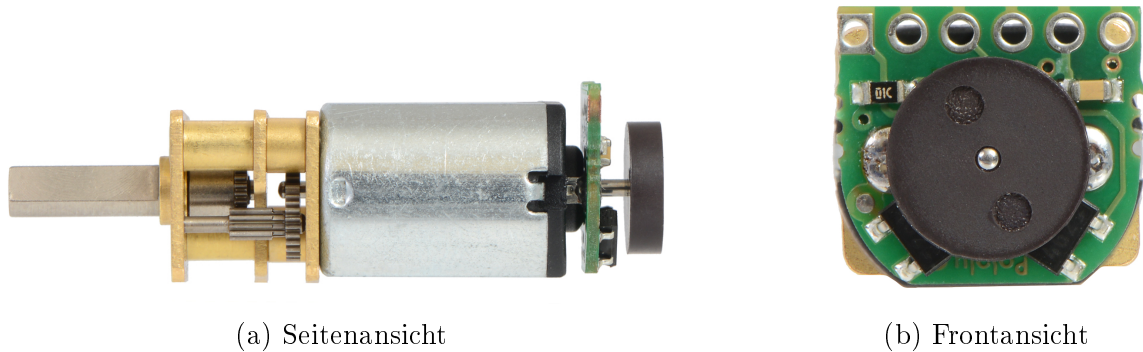


Abbildung 3.2: Exemplarische Encoder-Platine mit zwei Hall-Effekt-Sensoren in Quadratur [7]

Betrachtet man die in Abbildung 3.3 dargestellten Ausgangssignale, so fällt auf, dass die Flanken der Rechtecksignale zeitlich versetzt sind. Anhand der Reihenfolge der steigenden Flanken lässt sich die Drehrichtung deduzieren: Folgt der Impuls des zweiten Kanals dem des ersten, dreht sich die Welle in die eine Richtung; kehrt sich die Reihenfolge um, ändert sich die Drehrichtung. Aus der zeitlichen Differenz der aufeinanderfolgenden Flanken kann zudem die Drehgeschwindigkeit berechnet werden.

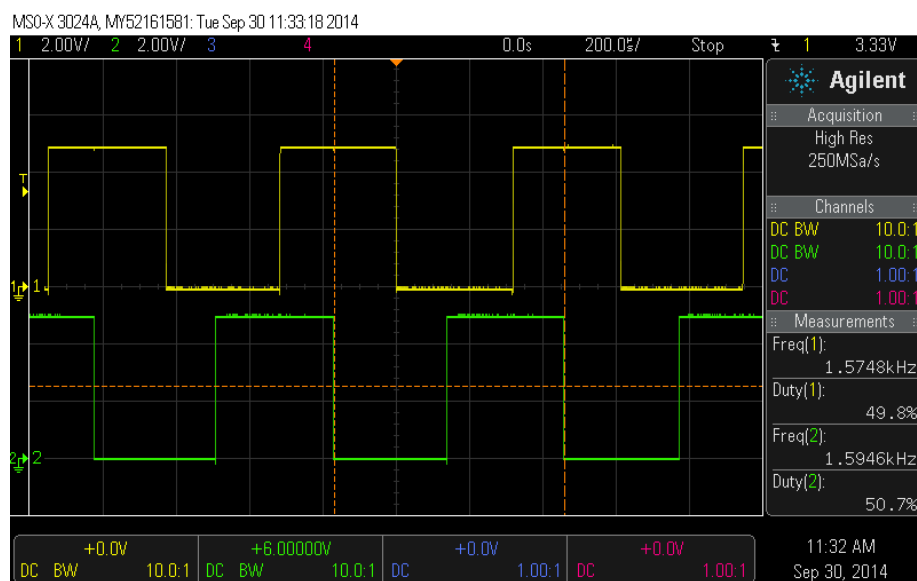


Abbildung 3.3: Exemplarisches Ausgangssignal eines Hall-Effekt-Sensors [7]

In der ersten Aufgabe des zweiten Übungsblattes sollen die Studierenden mithilfe der Encoder eine Strecke von 10 cm zurücklegen und diese anschließend physikalisch validieren (z. B. mit einem Lineal). Hierbei analysieren sie die Zusammenhänge zwischen Encoderwerten und realen Distanzen und vergleichen die gemessenen Ergebnisse mit den theoretischen Sollwerten. Zur Unterstützung wird auf die Header-Datei der Encoder im GitHub-Repository sowie auf die Dokumentation der Roboterdimensionen verwiesen. Es existieren mehrere Ansätze zur Lösung: Während ein empirisches Ermitteln der Encoderwerte möglich wäre, sollen die Studierenden durch den Verweis auf die Dimensionen gezielt zum analytischen Lösungsweg hingeführt werden.

Die Musterlösung gliedert sich in drei Teile. Im ersten Sketch wird die Distanz über die Encoder-Ticks berechnet. Dabei wird der Zusammenhang zwischen Reifendurchmesser und Getriebeübersetzung berücksichtigt. Mit der Formel für den Kreisumfang $U = \pi \cdot d$ und der Anzahl der Ticks pro Radumdrehung ($30 \times 12 = 360$) lässt sich die Distanz pro Tick mit folgender Gleichung bestimmen:

$$s_{\text{Tick}} = \frac{U}{N_{\text{Ticks}}} \quad (3.1)$$

Dabei bezeichnet s_{Tick} den Weg pro Encoder-Tick, U den Radumfang und N_{Ticks} die Anzahl der Ticks pro Radumdrehung.

Um nun die Anzahl der benötigten Ticks N zu ermitteln, die erforderlich sind, um eine bestimmte Strecke $s_{\text{gewünscht}}$ zurückzulegen, wird die gewünschte Distanz durch die pro Tick zurückgelegte Strecke dividiert. Daraus ergibt sich folgende Beziehung:

$$N = \frac{s_{\text{gewünscht}}}{s_{\text{Tick}}} \quad (3.2)$$

Durch Einsetzen der Werte lässt sich somit die erforderliche Anzahl an Encoderschritten für eine Distanz von 10 cm bestimmen. Eine ausführliche Berechnung mit konkreten Werten ist in der Musterlösung enthalten. Der ermittelte Wert stellt einen theoretischen Zielwert dar, der in der Praxis jedoch nur näherungsweise erreicht werden kann. Dies liegt an mehreren physikalischen und mechanischen Einflussfaktoren: Durch das Beschleunigen und Abbremsen des Roboters kommt es zu einem kurzzeitigen Durchrutschen der Reifen, welches von den Encodern nicht erfasst werden kann, da diese ausschließlich die Drehung der Motorwelle messen. Zudem verfügt der Roboter über lediglich zwei angetriebene Räder und eine stützende Kugel an der Rückseite, wodurch er bei Änderungen der Geschwindigkeit aufgrund seiner Trägheit leicht nach vorne oder hinten kippt. All diese Faktoren führen dazu, dass die angestrebte Distanz von 10 cm nicht exakt erreicht wird.

Zur Kompensation dieser Abweichung könnte eine Messreihe durchgeführt werden, deren durchschnittliche Abweichung in die Berechnung einfließt. Für den Rahmen der Übung ist dies jedoch nicht erforderlich, denn diese Ungenauigkeit bietet einen didaktischen Mehrwert: Die Studierenden erkennen, dass rein mathematische Modelle ohne Berücksichtigung von Störgrößen (systematische und zufällige Fehler) in der Realität oft limitiert sind.

Um diese Problemstellung zu vertiefen, wird in der zweiten Aufgabe die tatsächlich gefahrene Distanz mithilfe eines Infrarotsensors überprüft. Hierfür kommt der in Abbildung 3.4a dargestellte dreikanalige Time-of-Flight (ToF)-Sensor zum Einsatz, der auf dem OPT3101-IC basiert und ein Sichtfeld von ca. 180° abdeckt. Der Messbereich unterteilt sich in drei Kanäle (siehe Abbildung 3.4b). Zur Ansteuerung muss der gewünschte Kanal mittels `setChannel(uint8_t channel)` ausgewählt werden. Die Messung wird anschließend durch `sample()` gestartet, welche den Programmablauf blockiert, bis die Daten verfügbar sind. Die Funktion `readOutputRegs()` überträgt die Werte schließlich in die Membervariablen. Alternativ kann eine nicht-blockierende Messung via `startSample()` und `isSampleDone()` implementiert werden.

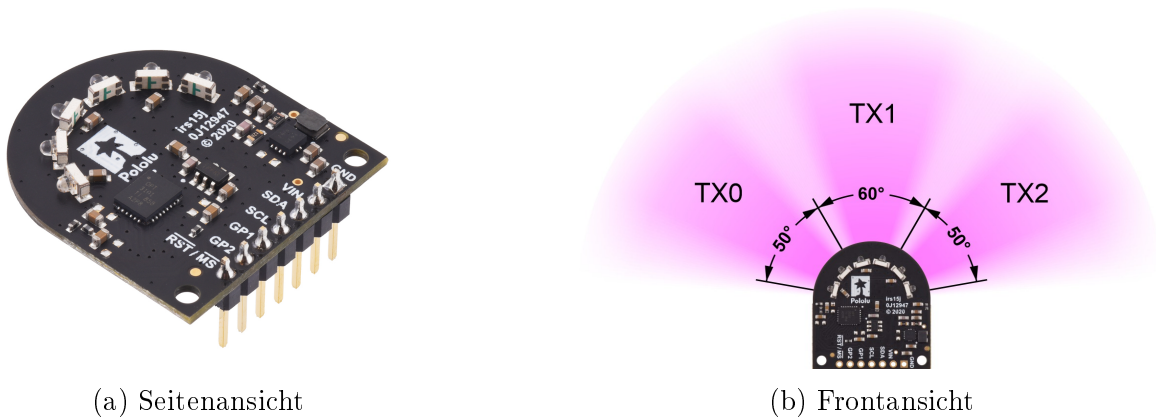


Abbildung 3.4: Infrarotsensorplatine und Aufteilung der Zonen [8]

Jede Zone verfügt über eine eigene Infrarot (IR)-LED und einen Empfänger. Die Messung erfolgt nach dem Time-of-Flight-Prinzip (vgl. Abbildung 3.5), bei dem ein kontinuierlich sinusförmig modulierte Infrarotsignal ausgesendet wird. Dieses wird vom Objekt reflektiert und gelangt mit einer Phasenverschiebung ϕ zum Sensor zurück, die durch die Laufzeit des Lichts entsteht. Zur Bestimmung der Phasenverschiebung wird das Empfangssignal mit zwei um 90° versetzten Referenzsignalen multipliziert, woraus sich zwei Komponenten ergeben:

- **i-Komponente** (In-Phase): Anteil des Signals, der mit dem ausgesendeten Signal in Phase ist.
- **q-Komponente** (Quadratur): Anteil des Signals, der um 90° phasenverschoben ist.

Aus diesen beiden orthogonalen Komponenten kann die Phasenverschiebung eindeutig berechnet werden:

$$\phi = \arctan 2(q, i) \quad (3.3)$$

wobei i und q die gemessenen Signalanteile darstellen. Diese Vorgehensweise vermeidet Mehrdeutigkeiten, die bei einer ausschließlichen Betrachtung von Sinus- oder Kosinusanteilen auftreten würden.

Aus der gemessenen Phasendifferenz ϕ lässt sich die Laufzeitdifferenz Δt mit der Modulationsfrequenz f_{mod} bzw. der Kreisfrequenz $\omega = 2\pi f_{\text{mod}}$ wie folgt bestimmen:

$$\Delta t = \frac{\phi}{\omega} \quad (3.4)$$

Setzt man diesen Zusammenhang in die allgemeine ToF-Gleichung ein, ergibt sich die Entfernung d zwischen Sensor und Objekt zu:

$$d = \frac{c_0 \cdot \Delta t}{2} = \frac{c_0 \cdot \phi}{2\omega}, \quad (3.5)$$

wobei $c_0 = 3 \times 10^8 \text{ m s}^{-1}$ die Lichtgeschwindigkeit in Luft ist.

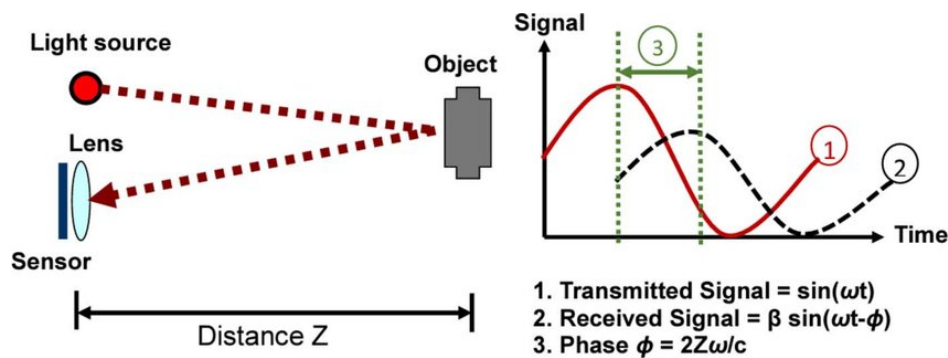


Abbildung 3.5: Time-of-Flight-Prinzip in Anwendung als Distanzsensor [9]

In der zweiten Aufgabe sollen die Studierenden den Code aus der vorherigen Übung so erweitern, dass der IR-Sensor die zurückgelegte Strecke überwacht. Dazu wird ein Hindernis in einer Entfernung von mehr als 10 cm platziert. Um die Bearbeitungszeit im Rahmen der 90 min zu halten, ist der Code für die `setup`-Funktion bereits vorgegeben. In diesem Code wird der Sensor initialisiert und das sogenannte Frame-Timing konfiguriert. Dabei wird festgelegt, wie viele Subframes pro Messung erfasst werden. Diese Anzahl beeinflusst die Messgenauigkeit, jedoch auch die Dauer einer Messung: Jede Subframe-Aufnahme dauert 0,25 ms. Der eingestellte Subframe-Count sollte eine Zweierpotenz sein. Für diese Übung wurde ein Wert von 4096 gewählt, was einer Messdauer von etwa 1 s entspricht.

Da die Messwerte lediglich zur Kontrolle und nicht zur aktiven Steuerung der zurückgelegten Strecke verwendet werden, ist eine längere Messdauer unkritisch, während eine höhere Genauigkeit vorteilhaft ist. Zusätzlich wird der zentrale Kanal des Sensors ausgewählt und die Helligkeit auf den Modus `Adaptive` gesetzt. Dies bewirkt, dass die Intensität der Infrarot-LEDs automatisch so angepasst wird, dass ein möglichst präzises Messergebnis erzielt wird, ohne dass der Sensor in die Sättigung gerät.

In der Musterlösung werden die oben beschriebenen Einstellungen in Form von Kommentaren erläutert und die entsprechenden Dokumentationen zu den verwendeten Bibliotheken verlinkt. In der `setup`-Funktion wird zunächst eine erste Messung durchgeführt, um die Ausgangsdistanz vor dem Start der Fahrt zu erfassen. In der

loop-Funktion wird anschließend, wie bereits in der vorherigen Aufgabe, die anvisierte Strecke von 10 cm mithilfe der Encoder-Klicks zurückgelegt. Nach dem Anhalten der Motoren bei Erreichen des Zielpunkts wird eine zweite Messung gestartet. Aus der Differenz zwischen der ersten und der zweiten Messung wird die tatsächlich zurückgelegte Strecke berechnet und das Ergebnis anschließend auf dem OLED-Display ausgegeben.

In der letzten Aufgabe des zweiten Übungsblatts sollen die Studierenden das in den vorherigen Aufgaben erworbene Wissen demonstrieren, indem sie mithilfe des IR-Sensors den Roboter so lange fahren lassen, bis dieser ein Hindernis erkennt und anschließend mit einer 90°-Drehung ausweicht. Diese Aufgabenstellung eignet sich besonders gut, um die erlernten Konzepte auf eine neue Problemstellung anzuwenden.

In der Musterlösung der ersten Aufgabe wurde gezeigt, wie sich anhand des Reifendurchmessers und der Anzahl der Encoder-Klicks Rückschlüsse auf die zurückgelegte Strecke ziehen lassen. Auf dieser Grundlage kann mithilfe einfacher mathematischer Zusammenhänge die Länge des Kreisbogens berechnet und daraus die erforderliche Anzahl an Encoder-Klicks für eine Drehung bestimmt werden. Alternativ kann die benötigte Drehzeit experimentell ermittelt und als feste Konstante im Programm hinterlegt werden. Dieser pragmatische Ansatz wurde auch in der Musterlösung gewählt.

Der Wissenstransfer aus der IR-Sensor-Aufgabe besteht darin, die Sensordaten während der Programmlaufzeit auszuwerten. Dazu muss ein geeigneter Zeitpunkt im Programmablauf definiert werden, an dem die Messung erfolgt. Falls, wie in der Musterlösung, die Konfiguration `OPT3101ChannelAutoSwitch` verwendet wird, müssen die Messwerte der verschiedenen Kanäle korrekt zugeordnet und ausgewertet werden.

In der Musterlösung wird auch die Problematik einer Anfahrt mit einem Winkel zur Wand angesprochen. Wird ein fester Schwellwert für den Frontsensor definiert, ab dem der Roboter ausweichen soll, kann es bei einer schrägen Anfahrt vorkommen, dass dieser Wert noch nicht erreicht wird, obwohl bereits eine Kollision mit der Wand stattgefunden hat. Dieses Problem lässt sich durch die Einbeziehung der seitlich angebrachten Sensoren beheben. Abbildung 3.6 zeigt diese Situation schematisch: Während der frontseitige IR-Sensor aufgrund des ungünstigen Winkels keine Rückreflexion empfängt, erkennt der seitliche Sensor die Wand.

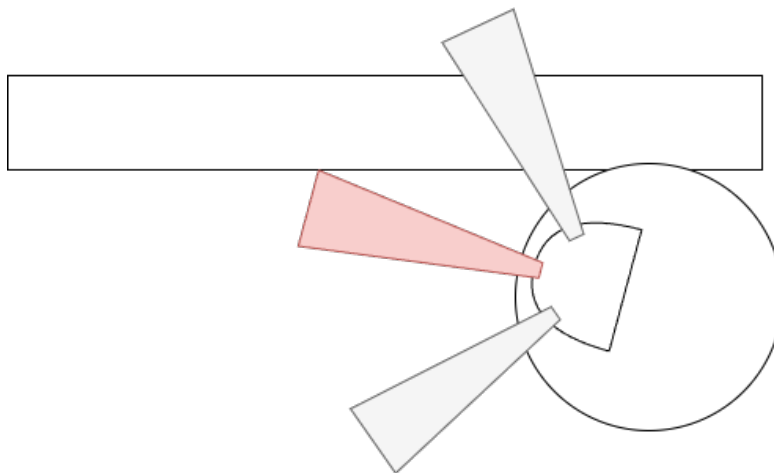


Abbildung 3.6: Schräger Anfahrwinkel führt dazu, dass der frontseitige IR-Sensor keine Rückreflexion empfängt, während der seitliche Sensor die Wand detektiert

Aus organisatorischer Sicht ist bei dieser Aufgabe auf eine strukturierte Testumgebung im Hörsaal zu achten, um gegenseitige Störungen der Roboter zu minimieren und einen geordneten Übungsablauf zu gewährleisten.

3.3 Übungsblatt 3: Implementierung eines Algorithmus zur Labyrinthlösung

Ziel der letzten Übungseinheiten ist die Implementierung einer Labyrinthlösung auf Grundlage einer vorgegebenen Codevorlage. Die Traversierung eines Labyrinths eignet sich aus mehreren Gründen besonders gut als Abschlussaufgabe, um die Kompetenzstufen der Anwendung und Synthese zu erreichen. Zum einen ist die Anschaulichkeit sehr hoch: Die Studierenden erhalten durch die Beobachtung des Roboters ein direktes Feedback, um das Ergebnis ihrer Programmierung zu validieren. Auch aus psychologischer Sicht ist diese Aufgabe zugänglich, da die meisten Studierenden bereits Erfahrungen mit dem Lösen eines klassischen Labyrinths gemacht haben (Anknüpfung an Vorwissen). Die Evaluation eines Roboters, der diese Aufgabe methodisch löst, fördert die Motivation und verdeutlicht den Transfer theoretischer Konzepte auf praktische Hard- und Software. Im Gegensatz zu abstrakten Sensordaten ist die Labyrinthlösung ein Ergebnis, das visuell überprüft werden kann.

Darüber hinaus erfordert die Aufgabe die Synthese aller zuvor eingeführten Sensor-komponenten. Auf diese Weise wird das in den vorangegangenen Übungen erworbene Wissen in einem konkreten Szenario angewendet, was den Transfer und die Reorganisation des Gelernten unterstützt. Zusätzlich lassen sich theoretische Inhalte aus der Vorlesung exemplifizieren. Im Rahmen der Informatikvorlesung werden grundlegende Datenstrukturen sowie Suchalgorithmen eingeführt, die für die Traversierung von

zentraler Bedeutung sind. Ein wesentliches theoretisches Fundament bildet hierbei die Graphentheorie. In der Graphentheorie bezeichnet ein Graph eine Menge von Knoten zusammen mit einer Menge von Kanten, wobei jede Kante ein Paar von Knoten zugeordnet ist [10]. Eine Kante ist hierbei eine Menge von genau zwei Knoten. Das Labyrinth kann als zusammenhängender ungerichteter Graph modelliert werden, wobei die Felder die Knoten sind und die möglichen Pfade die Kanten. Unter dieser Betrachtungsweise kann das Labyrinth mithilfe gängiger graphbasierter Suchalgorithmen systematisch gelöst werden.

Die in der Vorlesung eingeführte Tiefensuche Depth-First Search (DFS) eignet sich insbesondere aufgrund ihrer einfachen Implementierbarkeit und ihrer garantierten Vollständigkeit, das heißt, sie findet stets eine Lösung, sofern ein erreichbares Ziel existiert. In Wettbewerbsanwendungen ist jedoch häufig der Flood-Fill-Algorithmus verbreiteter, da die Tiefensuche zwar eine gültige Lösung liefert, jedoch nicht notwendigerweise den kürzesten Pfad identifiziert. Der Flood-Fill-Algorithmus hingegen verfolgt einen globaleren Ansatz und ermittelt systematisch stets die kürzeste Verbindung zwischen Start- und Zielpunkt, was insbesondere bei zeitoptimierten Labyrinthläufen einen entscheidenden Vorteil darstellt.

Bei der Tiefensuche in einem Graphen wird ausgehend vom Startknoten jeweils ein noch nicht besuchter Nachfolgerknoten aufgesucht. In der zugrunde liegenden Codevorlage wurde die Auswahl der Nachfolger so implementiert, dass zunächst der linke Nachbarknoten und anschließend alle weiteren im Uhrzeigersinn überprüft werden. Sobald eine Sackgasse erreicht ist, wird der bisherige Pfad mithilfe des sogenannten Backtrackings bis zu dem Knoten zurückverfolgt, von dem aus eine alternative Wahl möglich ist. Dadurch wird sichergestellt, dass kein Knoten und keine Kante mehrmals besucht werden [10].

In der Codevorlage wurde das Tracking der Nachbarknoten mithilfe eines `bool`-Arrays realisiert, in dem bereits besuchte Felder mit dem Wert `true` markiert werden. Die möglichen Wege wurden während der Durchfahrt mithilfe der Infrarotsensoren erfasst und in einem `int`-Array gespeichert. Die korrekte Zuordnung der erfassten Wege zu den jeweiligen Richtungen erfolgte über die globale Variable `headingDirection`, welche die aktuelle Ausrichtung des Roboters im Labyrinth fortlaufend speichert und bei Drehungen entsprechend aktualisiert wird. Das Backtracking wurde über rekursive Funktionsaufrufe umgesetzt, wodurch der Rückweg automatisch bis zu dem Knoten verfolgt wird, von dem aus eine alternative Route möglich ist.

Für die letzte Übungseinheit sollten bestimmte Funktionsdefinitionen der ursprünglichen Codevorlage entfernt und von den Studierenden eigenständig rekonstruiert und implementiert werden. Dabei erhielten die Studierenden ein Gerüst, das die Position des Roboters verfolgt. Ziel war es, dass die Studierenden dieses Grundgerüst zunächst eingehend analysieren und die Funktionsweise interpretieren, um anschließend eine Funktion zur Steuerung der Geradeausbewegung zu entwickeln. Diese muss gleichzeitig die Umgebung auf mögliche Wege scannen. Zudem sollten sie eine Backtracking-Funktion entwerfen, die die gesammelten Informationen auswertet und das Labyrinth traversiert.

Die ursprüngliche Codevorlage wurde von einer Gruppe von Schwerpunktstudenten erstellt, deren Aufgabe lediglich die Labyrinthlösung war. Infolgedessen waren die Funktionen nicht modular gekapselt und einige Variablen waren überflüssig. Nach sukzessivem Entfernen nicht benötigter Codeabschnitte und Verbesserung von Lesbarkeit und Kapselung zeigte sich jedoch, dass die Funktionslogik stark miteinander verflochten ist. Eine Abweichung vom Originalaufbau führte dazu, dass der Roboter den Code nicht zuverlässig ausführen konnte.

Eine zwingende Vorgabe würde den didaktischen Mehrwert der Übung eliminieren. Ziel der Übungsaufgabe ist es, dass die Studierenden eigenständig Abstraktionen bilden und Lösungsansätze entwickeln. Auf Grundlage dieser Erkenntnisse wurde entschieden, das Konzept der letzten Übung zu überarbeiten. Die Lösung des Labyrinths soll weiterhin Bestandteil sein, tritt jedoch nun als Demonstration auf. Studierende, die die regulären Aufgaben frühzeitig abschließen, können den fertigen Code analysieren, auf den Roboter laden und das Fahrverhalten evaluieren, um die Anwendung der gelernten Techniken zu verifizieren. Anstelle der ursprünglichen Aufgabe tritt die Inbetriebnahme eines weiteren Sensors (Gyroskop) und die Optimierung der Ergebnisse durch eine PID-Regelung.

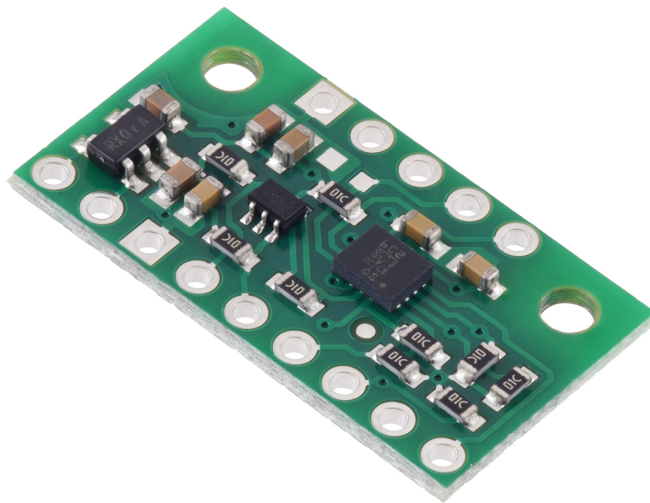


Abbildung 3.7: LSM6DSO Beschleunigungs- und Drehratensensor [11]

Auf dem Pololu 3pi+ 2040 ist der in Abbildung 3.7 dargestellte LSM6DSO als 3D-Beschleunigungs- und Drehratensensor verbaut. Solche Mikro-Elektromechanisches System (MEMS) basieren typischerweise auf einer beweglichen seismischen Masse, die eine Elektrode eines Kondensators bildet. Wird die Masse durch eine äußere Beschleunigung verschoben, ändert sich der Plattenabstand und somit die Kapazität des Kondensators (siehe Abbildung 3.8).

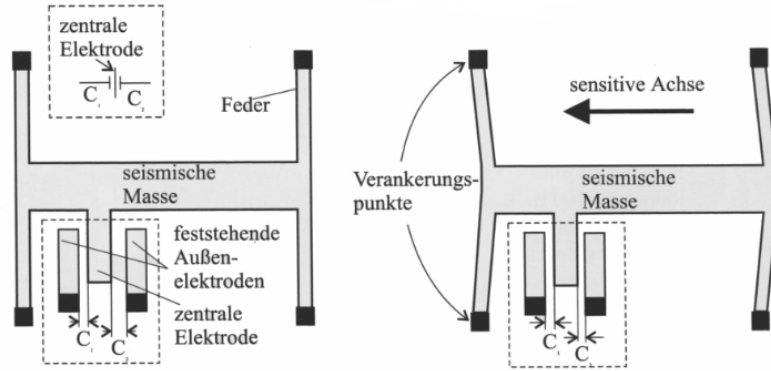


Abbildung 3.8: Schematische Darstellung eines Beschleunigungssensors [12]

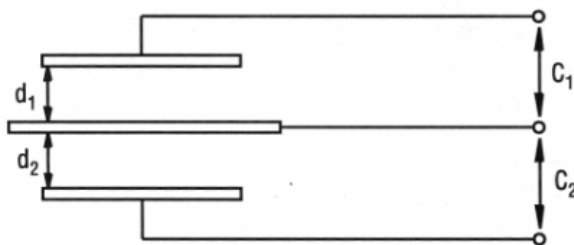
Das zugrundeliegende Messprinzip basiert auf dem Differentialkondensator, welcher in Abbildung 3.9a dargestellt ist. Die Änderung der Kapazität kann in Beziehung zur Änderung des Plattenabstands gesetzt werden gemäß:

$$f(C1, C2) = \frac{C_2 - C_1}{C_1 + C_2} = \frac{\Delta d}{d_0} \quad (3.6)$$

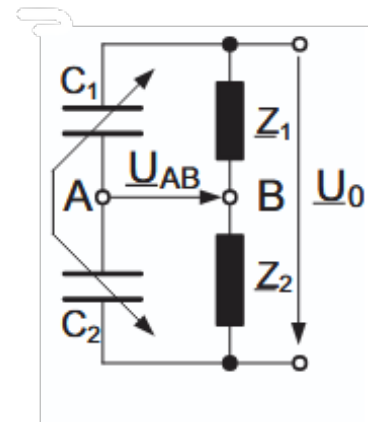
Über die Kapazitätsänderung lässt sich somit indirekt auf die Verschiebung der seismischen Masse und damit auf die wirkende Beschleunigung schließen. Werden die Kondensatoren in einer Brückenschaltung (siehe Abbildung 3.9b) verschaltet und die Impedanzen Z_0 und Z_1 gleich gewählt, kann die Kapazitätsänderung in eine proportionale Messspannung umgewandelt werden. Diese ergibt sich zu:

$$U_{AB} = \frac{U_0}{2} \cdot \frac{C_2 - C_1}{C_1 + C_2} = \frac{U_0}{2} \cdot \frac{\Delta d}{d_0} \quad (3.7)$$

Damit ergibt sich ein linearer Zusammenhang zwischen der Plattenverschiebung und der Ausgangsspannung der Brückenschaltung.



(a) Seitenansicht [12]



(b) Brückenschaltung [13]

Abbildung 3.9: Differentialkondensatorprinzip in Anwendung

Für den verwendeten Sensor existiert eine Header-Datei, die zahlreiche Schnittstellen bereitstellt und die erfassten Messdaten in einem direkt nutzbaren Format zur Verfügung stellt. Diese Header-Datei erfordert eine Instanz `buttonA` der Klasse `Buttons`, eine Instanz `display` der Klasse `Display` sowie eine Instanz `imu` der Klasse `IMU` (inertial measurement unit (IMU)).

Um die Nutzung des Gyroskops zu vereinfachen und die Erstellung zusätzlicher Objekte der Klassen `Display` und `Buttons` zu vermeiden, wurden sämtliche Verwendungen dieser Klassen aus der Header-Datei entfernt, ohne dabei die grundlegende Funktionalität zu beeinträchtigen.

Die Klasse `IMU` stellt eine direkte Repräsentation des Sensors selbst dar. Da die Header-Datei Funktionen dieser Klasse zur Implementierung ihrer eigenen Schnittstellen verwendet, muss sie nach der Instanziierung der Klasse eingebunden werden. Alternativ können die Funktionen der `IMU`-Klasse auch direkt genutzt werden. Im Rahmen der Übung ist es jedoch ausreichend, das Prinzip und die Verwendung der im Header bereitgestellten Funktionen zu verstehen.

Zur Initialisierung des Gyroskops sollten in der `setup()`-Funktion die Funktionen `turnSensorSetup()` und `turnSensorReset()` aufgerufen werden, um das Gyroskop zu kalibrieren und den Winkel zurückzusetzen. Während des Programmablaufs sollte die Funktion `turnSensorUpdate()` möglichst häufig aufgerufen werden, diese aktualisiert den Winkel und liest die Messdaten aus. Zur Verarbeitung der Messdaten steht die globale Variable `uint32_t turnAngle` zur Verfügung. Um daraus einen Winkel in Grad zu berechnen, kann folgende Umwandlung vorgenommen werden:

- Für den Bereich -180° bis 180° :

```
int32_t currentAngle = (((int32_t)turnAngle >> 16) * 360) >> 16;
```

- Für den Bereich 0° bis 360° :

```
uint32_t currentAngle = (((uint32_t)turnAngle >> 16) * 360) >> 16;
```

Mit den beschriebenen Funktionen und Variablen ist es möglich, eine Drehung um einen festgelegten Winkel zu realisieren. Darauf aufbauend soll nun das Prinzip der PID-Regelung angewendet werden. In der Regelungstechnik unterscheidet man grundsätzlich zwischen Steuern und Regeln. Das Steuern ist ein offener Wirkungsablauf, bei dem ein Eingangssignal ein Ausgangssignal beeinflusst, ohne dass Störungen ausgeglichen werden können [14]. Eine solche Steuerung wird strukturell als Steuerkette dargestellt.

Abbildung 3.10 zeigt eine Steuerkette am Beispiel der Distanzsteuerung des Roboters. Die Regelgröße $\mathbf{x}$ ist die zu beeinflussende Größe, hier die zurückgelegte Distanz, und entspricht dem Istwert. Die Führungsgröße $\mathbf{w}$ ist die Eingangsgröße und stellt den Sollwert dar, also die gewünschte Distanz. Die Steuereinrichtung erzeugt daraus die Stellgröße $\mathbf{u}$, beispielsweise eine Motorspannung, mit der das Stellglied die Regelgröße beeinflusst. Auf diesen Prozess wirken zudem äußere Störgrößen $\mathbf{z}$, welche das Ergebnis verändern können.

Hierdurch entsteht die Diskrepanz zwischen dem Code und den tatsächlich erzielten Ergebnissen. Zwar wird die Stellgröße korrekt von der Steuereinrichtung an das Stellglied übergeben, jedoch können externe Einflüsse wie Reifenschlupf oder Trägheit sowie interne Einflüsse wie der Batterieladungszustand die Regelgröße verändern und somit Abweichungen verursachen. Sind alle Störgrößen der Strecke bekannt, könnten diese durch eine angepasste Stellgröße kompensiert werden. Im konkreten Fall des Pololu 3pi+ 2040 ist dies jedoch praktisch kaum umsetzbar oder würde eine große Anzahl an Tests erfordern, um die Distanzabweichungen zu mitteln.

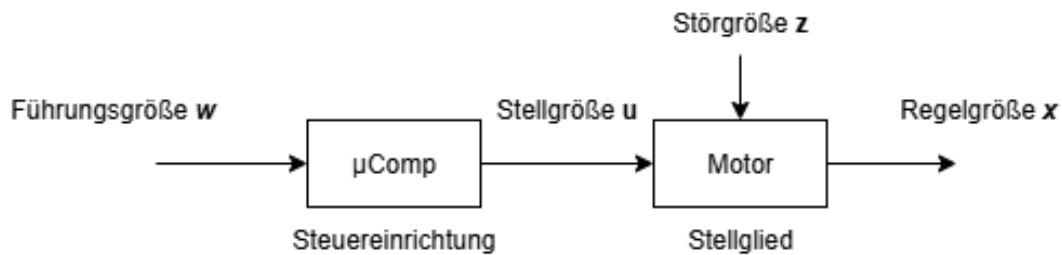


Abbildung 3.10: Einfache Steuerkette

Der Steuerung steht das Prinzip der Regelung gegenüber. Bei einer Regelung wird die Ausgangsgröße kontinuierlich erfasst, mit dem Sollwert verglichen und auf den Eingang zurückgeführt. Es entsteht ein geschlossener Wirkungsablauf, der sogenannte Regelkreis [14]. Durch die Rückführung der Regelgröße x und deren Subtraktion von der Führungsgröße entsteht die Regeldifferenz e , welche den Unterschied zwischen Soll- und Istwert beschreibt. Diese Differenz wird dem Regler als Eingangsgröße zugeführt, der daraus die Stellgröße erzeugt.

In Abbildung 3.11 ist dasselbe Szenario wie zuvor bei der Steuerkette dargestellt, jedoch so erweitert, dass eine Regelung umgesetzt wird. Der Regler ist in diesem Beispiel erneut der Mikrocontroller. In diesem wird die Regeldifferenz verarbeitet und in eine entsprechende Spannung für die Motoren umgewandelt. Der entscheidende Vorteil einer Regelung besteht darin, dass die Regelgröße gemessen und somit nachgeregelt werden kann. Dadurch kann das System besser mit Störgrößen umgehen, auch wenn es nicht vollständig gegen sie immun ist. Störungen treten dabei hauptsächlich bei der Erfassung des Istwertes auf. Die Messgenauigkeit kann von äußeren Einflüssen beeinträchtigt werden, so kann beispielsweise die Farbe eines Objekts die Genauigkeit eines IR-Sensors negativ beeinflussen. Auch interne Faktoren wirken sich auf die Messqualität aus, so hängt die Genauigkeit vom verwendeten Messglied ab: Ein Sensor mit geringer Dynamik kann keine ausreichend präzisen Aussagen über den Ausgang treffen, was zu Verzerrungen führt.

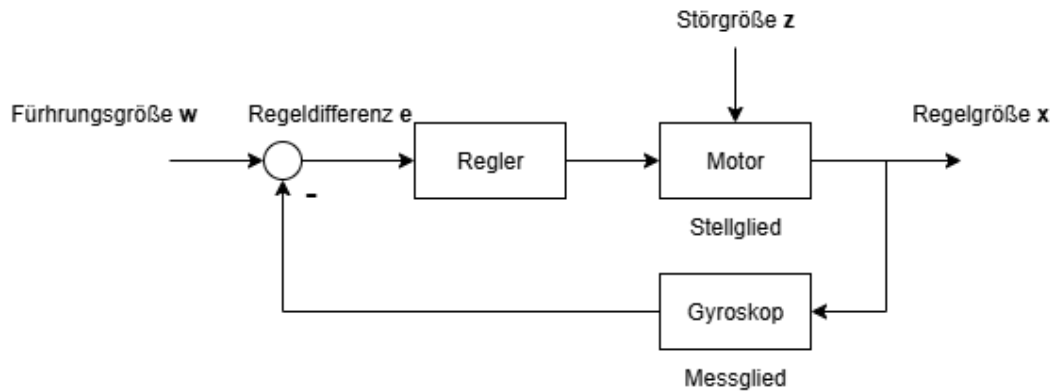


Abbildung 3.11: Einfache Regelstrecke

Im Folgenden werden drei wichtige Reglerarten näher erläutert, die in der Übung von den Studierenden angewendet werden sollen. Zudem enthält das Übungsblatt eine verkürzte Darstellung der PID-Regler-Theorie, um das Verständnis der Studierenden zu unterstützen.

Grundsätzlich wird zwischen stetigen und unstetigen Reglern unterschieden. Unstetige Regler können nur bestimmte, diskrete Betriebszustände einnehmen. Ein häufig verwendeter unstetiger Regler ist der Zweipunktregler, der lediglich zwischen „ein“ und „aus“ unterscheidet. Ein typisches Beispiel hierfür ist ein Kühlschrank: Er kühlt aktiv, bis die gewünschte Temperatur unterschritten wird, und schaltet anschließend ab, bis der obere Schwellenwert erneut erreicht ist.

Stetige Regler können die Stellgröße über einen kontinuierlichen Wertebereich innerhalb der zulässigen Grenzen annehmen. Der Regler arbeitet hierbei fortlaufend und passt die Stellgröße proportional zu den gemessenen Abweichungen an. Zu den stetigen Reglern zählen auch die PID-Regler. Die Abkürzung „PID“ steht dabei für die drei Teileanteile des Reglers: **p**roportional-, **i**ntegral- und **d**ifferential-Regler.

Im Folgenden werden diese drei Reglerarten genauer erläutert, da sie die Grundlage für den in dieser Übung einzusetzenden PID-Regler bilden. Jede der drei Komponenten trägt auf unterschiedliche Weise zur Dynamik und Stabilität des Regelkreises bei.

Proportional-Regler (P-Regler) Der Proportional-Regler erzeugt eine Ausgangsgröße, die proportional zur Eingangsgröße ist. Die Eingangsgröße entspricht hier dem Regelfehler, also der Differenz zwischen Soll- und Istwert. Der P-Regler reagiert schnell und stellt unmittelbar eine Stellgröße bereit. Allerdings verbleibt in der Regel eine stationäre Regeldifferenz. Die Stellgröße lässt sich durch folgende Gleichung beschreiben:

$$u(t) = K_P \cdot e(t) \quad (3.8)$$

Integral-Regler (I-Regler) Der Integral-Regler bildet am Ausgang die zeitliche Summe der Eingangsgröße, also des Regelfehlers, ab. Dadurch wirkt er langsam, kann jedoch eine verbleibende Regeldifferenz vollständig ausgleichen. Der I-Anteil wird durch folgende Gleichung beschrieben:

$$u(t) = K_I \cdot \int_0^t e(\tau) d\tau \quad (3.9)$$

Differential-Regler (D-Regler) Der Differential-Regler reagiert auf die Änderungsrate des Regelfehlers. Er wirkt dämpfend und verhindert ein überschießendes Systemverhalten, indem er schnelle Veränderungen im Fehler erkennt und diesen entgegenwirkt. Ähnlich wie der Proportional-Regler erzeugt auch der D-Regler nur dann ein Ausgangssignal, wenn sich der Fehler verändert. Bleibt der Fehler konstant, ist die Änderungsrate null und der D-Regler liefert kein Stellgrößensignal. Daher kann der D-Anteil eine bleibende Regeldifferenz nicht ausgleichen; er dient ausschließlich der Stabilisierung und Dämpfung des Systems. Mathematisch entspricht seine Wirkung der zeitlichen Ableitung des Fehlers:

$$u(t) = K_D \cdot \frac{de(t)}{dt} \quad (3.10)$$

In Abbildung 3.12 ist der Regelkreis mit dem PID-Regler integriert dargestellt.

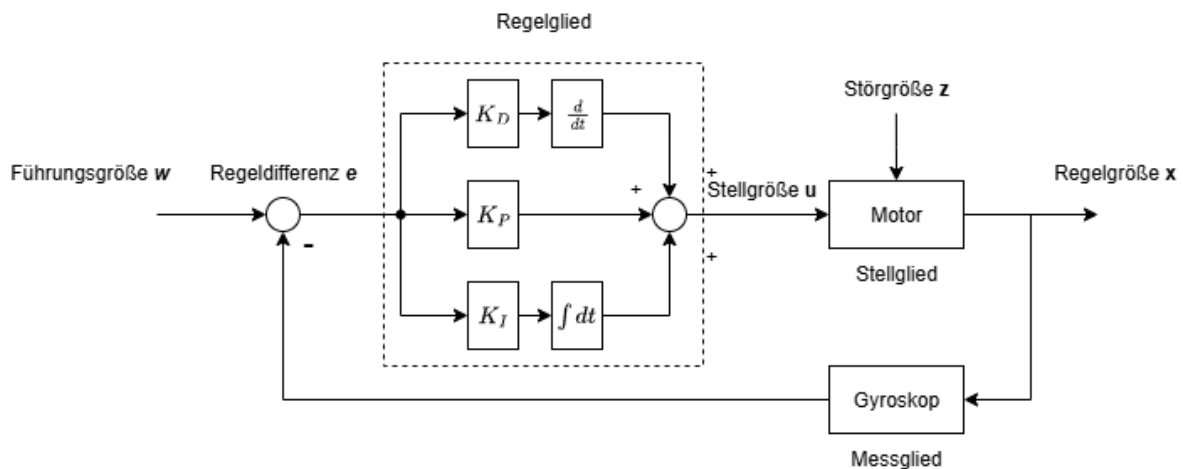


Abbildung 3.12: PID-Regler im Regelkreis

Durch die Kombination aller drei Regleranteile lässt sich das Verhalten des Systems gezielt beeinflussen. Der P-Anteil sorgt für eine schnelle Reaktion, der I-Anteil eliminiert bleibende Abweichungen, und der D-Anteil dämpft das System und reduziert Überschwingen. Die Stellgröße wird durch folgende Formel beschrieben:

$$u(t) = K_P \cdot e(t) + K_I \cdot \int_0^t e(\tau) d\tau + K_D \cdot \frac{de(t)}{dt} \quad (3.11)$$

Die Wahl der richtigen Gewichtung der Parameter K_P , K_I und K_D ist von zentraler Bedeutung. Eine ungeeignete Parametrierung kann zu einer Übersteuerung führen und das System im Extremfall sogar beschädigen. Ebenso kann eine falsche Einstellung dazu führen, dass die Regelung nicht korrekt arbeitet und der Sollwert nicht erreicht wird. Grundsätzlich existieren zwei Ansätze zur Bestimmung dieser Parameter: Zum einen können sie theoretisch berechnet werden. Dies setzt jedoch voraus, dass sämtliche relevanten Systemparameter, wie etwa der Reifenschlupf, die Motoreffizienz oder Reibungsverluste, bekannt sind. Da diese Größen im vorliegenden Fall nicht vollständig erfasst werden können, ist dieser Ansatz praktisch nicht umsetzbar. Der zweite Ansatz beruht auf der empirischen Bestimmung der Parameter. Hierfür existieren verschiedene Verfahren, wie beispielsweise die Methode nach Ziegler und Nichols. Im Rahmen dieser Übung wurden die Parameter empirisch ermittelt und den Studierenden zur Verfügung gestellt. Dies dient dazu, den zeitlichen Rahmen einzuhalten, da eine eigenständige Parametrierung ein tieferes Verständnis auf der Stufe der Synthese und Evaluation erfordern würde, welches über die angestrebten Lernziele dieser spezifischen Übungseinheit hinausgeht.

Nach der Implementierung der PID-Regelung und einem Vergleich mit der zuvor untersuchten Drehung ohne PID-Regler wurde festgestellt, dass die Unterschiede im Verhalten des Systems zu gering ausfielen, um den Studierenden den praktischen Nutzen einer Regelung nachvollziehbar zu vermitteln. Die Ursache für diesen geringen Unterschied könnte sowohl in der hohen Genauigkeit des verwendeten Gyroskops als auch in der generell geringeren Störanfälligkeit einer Drehbewegung liegen.

Um den Vorteil einer Regelung deutlicher hervorzuheben, wurde deshalb entschieden, die Aufgabe anzupassen: Statt die PID-Regelung auf eine Drehung anzuwenden, wird nun eine Vorwärtsbewegung verwendet, in Kombination mit einem deutlich störanfälligeren Infrarotsensor.

Die neue Aufgabenstellung besteht darin, den Roboter zunächst ohne PID-Regelung bis auf 10 cm an ein Hindernis heranzufahren. Hierzu wird der Abstand kontinuierlich gemessen und der Roboter bremst, sobald der Schwellwert unterschritten wird. Aufgrund der Verzögerung des Systems und der Massenträgheit wird der Roboter jedoch zu nahe an der Wand zum Stillstand kommen. Der tatsächlich erreichte Abstand wird anschließend auf dem OLED-Display angezeigt.

Nach einer kurzen Wartezeit startet der Programmteil mit aktivierter PID-Regelung. Dies wird für die Studierenden sichtbar, indem der Roboter zunächst ein Stück rückwärts von der Wand wegfährt und dann präzise bei einer Distanz von 10 cm zum Stillstand kommt. Durch die Einbeziehung des Regelfehlers können systematische Abweichungen, wie etwa Verzögerungen beim Bremsen oder die Trägheit des Roboters – kompensiert werden. Diese Gegenüberstellung macht für die Lernenden anschaulich nachvollziehbar, warum Regelungen in der Praxis eingesetzt werden und welchen Mehrwert sie bei realen Problemstellungen bieten.

Die finale Version des letzten Übungsblattes gliedert sich in drei Aufgabenbereiche. Zunächst implementieren die Studierenden die PID-Regelung. Dabei nutzen sie die im Aufgabenblatt bereitgestellten Erläuterungen, um die theoretischen Grundlagen sowie die physikalischen Auswirkungen der Parameter für die Programmierung nachzuvollziehen. Danach wird das Gyroskop als neue Sensorkomponente eingeführt und angewendet, um eine kontrollierte 90°-Drehung des Roboters zu realisieren. Für besonders leistungsstarke Studierende ist eine differenzierende Abschlusssaufgabe vorgesehen, welche die Analyse des vollständigen Quellcodes sowie die Evaluation der Lösungsstrategie anhand der praktischen Ausführung umfasst.

In der Musterlösung gibt es Folgendes zu beachten: Die oben dargestellten Gleichungen beschreiben die Regler im Zeitbereich. Aufgrund der digitalen Natur unseres Systems müssen sie jedoch als digitale Regler formuliert werden. Es liegen keine kontinuierlichen Signale am Regler an, sondern diskrete Abtastwerte, die mit dem Index k bezeichnet werden. Die Gleichungen ändern sich daher wie folgt:

P-Regler

$$u_k = K_P \cdot e_k \quad (3.12)$$

I-Regler

$$u_k = K_I \cdot T_A \cdot \sum_{i=0}^k e_i \quad (3.13)$$

D-Regler

$$u_k = K_D \cdot \frac{e_k - e_{k-1}}{T_A} \quad (3.14)$$

Hierbei entspricht T_A der Abtastperiode. Diese wurde aus der Anzahl der verwendeten Subframes des IR-Sensors und dem zeitlichen Abstand zwischen den PID-Aufrufen berechnet. Für die Implementierung wird die Summe der Fehler für den I-Anteil in einem globalen Array abgelegt, damit der Wert über die einzelnen Schleifendurchläufe hinweg erhalten bleibt. Für den D-Anteil wird hingegen der Fehler aus dem vorherigen Durchlauf in einer globalen Variable gespeichert, um im nächsten Schritt die Differenz zum aktuellen Fehler berechnen zu können. Nachdem alle Regleranteile addiert wurden, wird die Stellgröße mit der Funktion `constrain()` auf den zulässigen Geschwindigkeitsbereich begrenzt. Die minimale Geschwindigkeit wurde experimentell mit einem Wert von 45 bestimmt, da die Motoren bei kleineren Werten nicht mehr reagieren.

In der Musterlösung zur Drehung mit dem Gyroskop wird in einer `while(true)`-Schleife das Gyroskop kontinuierlich über die Funktion `turnSensorUpdate()` aktualisiert und der aktuelle Fehler berechnet. Liegt der Fehler innerhalb der festgelegten Toleranz, wird die Schleife beendet. Andernfalls wird der Fehler, analog zur vorherigen Aufgabe, mithilfe von `constrain()` auf den gültigen Geschwindigkeitsbereich abgebildet.

4 Zukünftiges Konzept für den Aufbau einer Micromouse-Infrastruktur

Zukünftig könnte eine Erweiterung der Micromouse-Infrastruktur angestrebt werden, um noch mehr Studierende für die Themen Robotik und Algorithmik zu begeistern. Im Rahmen von nur drei Übungen, die als Einführung für Studierende gedacht sind, die bisher keine Erfahrung mit der Programmierung von Robotern haben, kann jedoch nicht in die Tiefe gegangen werden, sodass das volle Potenzial und die eigentliche Komplexität des Micromouse-Wettbewerbs kaum ausgeschöpft werden können.

Da die grundlegende Infrastruktur nun vorhanden ist, eröffnen sich mehrere mögliche Wege. Zum einen könnte der Fokus stärker auf die Hardware gelegt werden, etwa indem ein eigener Roboter konzipiert wird. Online existieren bereits Vorlagen für Platinen, jedoch können, je nach gewünschtem Lernziel, auch komplett eigene Designs entwickelt werden, insbesondere wenn ein wirklich tiefes Verständnis bei den Studierenden gefördert werden soll. Darüber hinaus gibt es zahlreiche Ansatzpunkte für Verbesserungen, wie beispielsweise das Anbringen von kleinen Lüftern, die durch Unterdruck für höhere Kurvengeschwindigkeiten sorgen. Solche Erweiterungen bieten viel Raum für Kreativität und motivieren Studierende, eigene Lösungen zu erarbeiten.

Der Fokus könnte alternativ auch auf die Softwareseite gelegt werden, etwa in Form experimenteller Versuche mit unterschiedlichen Algorithmen oder der Optimierung bestehender Verfahren. Dies setzt allerdings ein deutlich tieferes Verständnis und eine größere zeitliche Investition voraus. Daher wäre es sinnvoll, ein solches Vorhaben im Rahmen eines Wahlpflichtfachs umzusetzen. Falls die Möglichkeit besteht, könnte dies sogar interdisziplinär in Zusammenarbeit mit dem Robotiklabor realisiert werden.

Literatur

- [1] Wikipedia. *Micromouse*. 2025. URL: <https://en.wikipedia.org/wiki/Micromouse> (besucht am 04.08.2025).
- [2] Ritesh Misra und Richard Adler. *Micromouse Competition Rules*. 2018. URL: <https://ewh.ieee.org/reg/2/sac-18/MicromouseRules.pdf> (besucht am 04.08.2025).
- [3] Wikipedia. *Micromouse Maze*. 2025. URL: https://de.wikipedia.org/wiki/Micromouse#/media/Datei:Micromouse_maze.jpg (besucht am 05.10.2025).
- [4] Pololu Corporation. *Pololu 3pi+ User Guide*. 2023. URL: <https://www.pololu.com/docs/0J86> (besucht am 04.08.2025).
- [5] jbabio. *3D printable Micromouse labyrinth Walls & Connectors*. 2025-10-19. URL: <https://www.thingiverse.com/thing:2594629> (besucht am 05.10.2025).
- [6] Pololu Corporation. *Encoder Product Page*. o.D. URL: <https://www.electronics-tutorials.ws/electromagnetism/hall-effect.html> (besucht am 05.10.2025).
- [7] Pololu Corporation. *Magnetic Encoder Pair Kit for Micro Metal Gearmotors*. Abbildung entnommen von der Produktseite, exemplarisch zur Veranschaulichung des Hall-Effekt-Encoders in Quadratur. o.D. URL: <https://www.pololu.com/product/3081> (besucht am 05.10.2025).
- [8] Pololu Corporation. *3-Channel Wide FOV Time-of-Flight Distance Sensor*. Abbildung entnommen von der Produktseite, exemplarisch zur Veranschaulichung des ToF Sensors. o.D. URL: <https://www.pololu.com/product/3680/pictures> (besucht am 05.10.2025).
- [9] Cyrus Bamji u. a. „A 0.13 μm CMOS System-on-Chip for a 512×424 Time-of-Flight Image Sensor with Multi-Frequency Photo-Demodulation up to 130 MHz and 2 GS/s ADC“. In: *IEEE Journal of Solid-State Circuits* 50 (Jan. 2015), S. 303–319. DOI: 10.1109/JSSC.2014.2364270.
- [10] Hartmut Ernst, Jochen Schmidt und Gerd Beneken. „Datenstrukturen, Bäume und Graphen“. In: *Grundkurs Informatik: Grundlagen und Konzepte für die erfolgreiche IT-Praxis – Eine umfassende Einführung*. Wiesbaden: Springer Fachmedien Wiesbaden, 2023, S. 631–712. ISBN: 978-3-658-41779-6. DOI: 10.1007/978-3-658-41779-6_14. URL: https://doi.org/10.1007/978-3-658-41779-6_14.
- [11] Pololu Corporation. *LSM6DSO Product Page*. 2023. URL: <https://www.pololu.com/product/2798> (besucht am 04.08.2025).
- [12] Thielemann. *Vorlesungsskript zur Veranstaltung SSensoren und Aktoren*. Unveröffentlichtes Skript, Hochschule Aschaffenburg. 2025.
- [13] Ansgar Meroth und Petre Sora. „Beschleunigungssensoren“. In: *Sensornetzwerke in Theorie und Praxis: Embedded Systems-Projekte erfolgreich realisieren*. Wiesbaden: Springer Fachmedien Wiesbaden, 2021, S. 427–453. ISBN: 978-3-658-31709-6. DOI: 10.1007/978-3-658-31709-6_16. URL: https://doi.org/10.1007/978-3-658-31709-6_16.

- [14] Thomas Beier und Petra Wurl. *Regelungstechnik*. 3., aktualisierte Auflage. München: Carl Hanser Verlag GmbH & Co. KG, 2022. DOI: 10.3139/9783446474031. eprint: <https://www.hanser-elibrary.com/doi/pdf/10.3139/9783446474031>. URL: <https://www.hanser-elibrary.com/doi/abs/10.3139/9783446474031>.